

معماری پایگاه داده محصولات متغیر در داتانت و انتیتی فریمورک کور

۲۶ شهریور ۱۴۰۵

فهرست مطالب

۱	خلاصه اجرایی و هدف سند	۱
۱	معماری بنیادین سیستم: الگوی SPU و SKU	۲
۲	پوشش حالت‌های چهارگانه قیمت‌گذاری و مدیریت موجودی	۳
۳	سناریوی عملی: داده‌های نمونه ست لباس در جداول دیتابیس	۴
۳	۱.۴ جدول ۱: Attributes (ویژگی‌ها)	۳
۳	۲.۴ جدول ۲: AttributeValues (مقادیر ویژگی‌ها)	۳
۳	۳.۴ جدول ۳: Products (محصولات)	۳
۳	۴.۴ جدول ۴: ProductVariants (واریانت‌ها / SKU)	۳
۳	۵.۴ جدول ۵: ProductVariantValues (جدول واسط)	۳
۴	۶.۴ سناریوی انتخاب کالا توسط کاربر	۴
۴	۵ تحلیل وابستگی ویژگی‌ها و مدیریت بهینه متادیتا (کد رنگ و مشخصات فنی)	۴
۴	۱.۵ ۱. وابستگی سایز به رنگ در سطح دیتابیس: منتفی است	۴
۴	۲.۵ ۲. سه راهکار مدیریت بهینه متادیتا	۴
۵	۶ عملیات پنل مدیریت: عدم اجبار در ترکیب‌ها و الگوی تولید ماتریس تنوع	۵
۵	۱.۶ ۱. آیا فروشنده مجبور است به ازای هر رنگ تمام سایزها را وارد کند؟	۵
۵	۲.۶ ۳. آیا ادمین باید قیمت‌های تکراری را دستی وارد کند؟	۵
۵	۳.۶ ۴. نکته تکمیلی: فیلد قیمت نال‌پذیر	۵
۶	۷ جمع‌بندی و توصیه‌های پیاده‌سازی	۶

۱ خلاصه اجرایی و هدف سند

این سند، جمع‌بندی کامل گفت‌وگوهای فنی میلاد درباره طراحی معماری پایگاه داده انعطاف‌پذیر برای فروشگاه اینترنتی در بستر داتانت و انتیتی فریمورک کور (EF Core) است. مسئله اصلی، مدیریت محصولات با تنوع‌های مختلف مانند طلا (بر اساس وزن) و پوشاک (بر اساس سایز و رنگ) بدون نیاز به تغییر اسکیمای دیتابیس در آینده است. راهکار پیشنهادی، الگوی SPU/SKU است که در آن موجودیت

‘ProductVariant به عنوان واحد انبارداری و فروش عمل می‌کند. این ساختار به صورت ذاتی هر چهار حالت قیمت‌گذاری (عادی، فقط رنگ، فقط سایز، سایز به همراه رنگ) را پوشش می‌دهد و با الگوی Matrix Generator در پنل مدیریت، از ورود دستی و تکراری قیمت‌ها جلوگیری می‌کند.

۲ معماری بنیادین سیستم: الگوی SKU و SPU

پیشنهاد اصلی این معماری، تفکیک دو مفهوم بنیادین در خرده‌فروشی است: SPU به عنوان موجودیت کلی محصول و SKU به عنوان واحد نهایی قابل فروش. طبق الگوی رایج در صنعت، قیمت و موجودی در سطح SKU نگهداری می‌شود، زیرا تنوع‌های مختلف یک محصول می‌توانند قیمت‌های متفاوتی داشته باشند (به عنوان مثال، سایز XXL ممکن است به دلیل مصرف پارچه بیشتر گران‌تر باشد). این الگو توسط توسعه‌دهندگانی مانند مارتین بین در پروژه‌های تجاری متعدد به کار رفته و اثبات شده است.

ساختار داده‌ای پیشنهادی شامل پنج موجودیت اصلی است:

```
public class Attribute : Base
{
    public string Name { get; set; } = string.Empty; // " ", " "
    public virtual ICollection<AttributeValue> Values { get; set; } = new List<AttributeValue>();
}

public class AttributeValue : Base
{
    public int AttributeId { get; set; }
    public string Value { get; set; } = string.Empty; // " ", "L"
    public string? DisplayColorHex { get; set; } //
    public virtual Attribute? Attribute { get; set; }
}

public class Product : Base
{
    public string Title { get; set; } = string.Empty;
    public PricingType PricingType { get; set; }
    public decimal? BasePrice { get; set; }
    public int? BaseStock { get; set; }
    public virtual ICollection<ProductVariant> Variants { get; set; } = new List<ProductVariant>();
}

public class ProductVariant : Base
{
    public int ProductId { get; set; }
    public string Sku { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public int StockQuantity { get; set; }
    public virtual Product Product { get; set; } = null!;
    public virtual ICollection<ProductVariantValue> VariantValues { get; set; } = new List<ProductVari
}

public class ProductVariantValue
{
    public int ProductVariantId { get; set; }
```

```

public virtual ProductVariant ProductVariant { get; set; } = null!;
public int AttributeValueId { get; set; }
public virtual AttributeValue AttributeValue { get; set; } = null!;
}

```

در این ساختار، ProductVariantValue به عنوان جدول واسط عمل می کند که هر SKU را به مقادیر ویژگی های مرتبط متصل می سازد. این اتصال چرخه ای است و نیازی به وابستگی سلسله مراتبی بین ویژگی ها (مانند وابستگی سایز به رنگ) در سطح دیتابیس نیست.

۳ پوشش حالت های چهارگانه قیمت گذاری و مدیریت موجودی

ساختار پیشنهادی به صورت ذاتی هر چهار سناریوی قیمت گذاری را پوشش می دهد:

۱. **محصول ساده:** دارای PricingType برابر Simple، به همراه BasePrice و BaseStock در جدول Product. هیچ واریانته ای ایجاد نمی شود.
۲. **محصول تک ویژگی (فقط رنگ یا فقط سایز):** دارای PricingType برابر VariantBased. هر SKU تنها به یک مقدار ویژگی متصل می شود.
۳. **محصول چندویژگی (سایز + رنگ):** هر SKU به دو یا چند مقدار ویژگی متصل می شود که در جدول ProductVariantValue ثبت می گردد.
۴. **محصول بدون تنوع قیمتی اما با تنوع موجودی:** قیمت در سطح SKU مستقل نگهداری می شود تا در آینده امکان تغییر قیمت برای یک تنوع خاص بدون تاثیر بر سایر SKU ها فراهم باشد.

فیلدهای BasePrice و BaseStock به صورت نال پذیر تعریف شده اند تا فقط برای محصولات ساده مقداردهی شوند و در محصولات متغیر، قیمت گذاری از طریق واریانته ها انجام گیرد.

۴ سناریوی عملی: داده های نمونه ست لباس در جداول دیتابیس

برای درک بهتر عملکرد این معماری، سناریوی ثبت یک ست بلوز و شلوار زنانه با کد A1115 را بررسی می کنیم. این محصول دارای دو رنگ (سرمه ای و کرم) و دو سایز (L و XL) است.

۱.۴ جدول ۱: Attributes (ویژگی ها)

Name	Id
رنگ	1
سایز	2

۲.۴ جدول ۲: AttributeValues (مقادیر ویژگی ها)

DisplayColorHex	Value	AttributeId	Id
#000080	سرمه ای	1	10
#F5F5DC	کرم	1	11
NULL	L	2	20

DisplayColorHex	Value	AttributeId	Id
NULL	XL	2	21

۳.۴ جدول ۳: Products (محصولات)

BaseStock	BasePrice	PricingType	Title	Id
NULL	NULL	VariantBased	ست بلوز و شلوار زنانه کد A1115	100

۴.۴ جدول ۴: ProductVariants (واریانت‌ها / SKU)

StockQuantity	Price	SKU	ProductId	Id
5	899,000	SET-A1115-NAVY-L	100	501
2	899,000	SET-A1115-NAVY-XL	100	502
0	920,000	SET-A1115-CREAM-L	100	503
8	950,000	SET-A1115-CREAM-XL	100	504

۵.۴ جدول ۵: ProductVariantValues (جدول واسط)

AttributeValueId	ProductVariantId
10	501
20	501
10	502
21	502
11	503
20	503
11	504
21	504

۶.۴ سناریوی انتخاب کالا توسط کاربر

۱. کاربر وارد صفحه محصول می‌شود و دو دایره رنگی (سرمه‌ای و کرم) را مشاهده می‌کند.
۲. با کلیک روی رنگ «کرم»، سیستم واریانت‌های متصل به AttributeValueId برابر ۱۱ را فیلتر می‌کند: سایز L ناموجود (موجودی صفر) و سایز XL موجود با ۸ عدد.
۳. با انتخاب سایز XL، واریانت ۵۰۴ هدف‌گیری شده و قیمت ۹۵۰,۰۰۰ تومان به همراه دکمه فعال «افزودن به سبد خرید» نمایش داده می‌شود.
۴. در صورت تغییر رنگ به «سرمه‌ای»، واریانت ۵۰۲ پیدا شده و قیمت به صورت خودکار به ۸۹۹,۰۰۰ تومان تغییر می‌کند.

۵ تحلیل وابستگی ویژگی‌ها و مدیریت بهینه متادیتا (کد رنگ و مشخصات فنی)

دیدن ستون DisplayColorHex در جدول عمومی AttributeValue ممکن است این حس را ایجاد کند که «رنگ» به ریشه همه ویژگی‌ها چسبیده است. نکته کلیدی این است که این دو موضوع باید از یکدیگر تفکیک شوند:

۱.۵ ۱. وابستگی سایز به رنگ در سطح دیتابیس: منتفی است

در دنیای واقعی انبارداری، «رنگ قرمز» و «سایز L» دو مفهوم کاملاً مستقل هستند و هیچ‌کدام فرزند دیگری نیستند. چیزی که این دو را به هم گره می‌زند، ترکیب آن‌ها در قالب یک کالای فیزیکی (SKU) است. سلسله‌مراتب نمایشی مانند «ابتدا رنگ را انتخاب کن، سپس سایزهای موجود آن رنگ را ببین» صرفاً یک موضوع UI/UX در فرانت‌اند است، نه یک وابستگی در پایگاه داده. دیتابیس نباید بگوید «سایز فرزند رنگ است»، زیرا ممکن است در محصول دیگری (مانند کفش)، مشتری ابتدا سایز ۴۲ را انتخاب کند و سپس رنگ‌های موجود آن سایز را ببیند.

۲.۵ ۲. سه راهکار مدیریت بهینه متادیتا

برای جلوگیری از آلوده شدن جدول AttributeValue با فیلدهای متفرقه، سه راهکار وجود دارد:

راهکار اول: ستون JSON Metadata (مدرن‌ترین در EF Core)

```
public class AttributeValue : Base
{
    public int AttributeId { get; set; }
    public string Value { get; set; } = string.Empty;
    // : {"hex": "#FF0000"}
    // : {"unit": "gram", "purity": 18}
    // : null
    public string? ExtraDataJson { get; set; }
    public virtual Attribute? Attribute { get; set; }
}
```

در EF Core 8 و 9 می‌توان این فیلد را به‌عنوان Complex Type مپ کرد و داخل LINQ به‌سادگی کوئری زد.

راهکار دوم: الگوی وراثت TPT/TPH

تعریف یک موجودیت پایه انتزاعی (SimpleAttributeValue) و یک کلاس مشتق‌شده برای رنگ (ColorAttributeValue) که فیلد تایپ‌سیف HexCode را دارد. این روش برای معماری‌هایی که اهمیت زیادی برای Type-Safety قائل‌اند مناسب است.

راهکار سوم: نگه‌داشتن ستون با مدیریت نوع ویژگی

ستون DisplayColorHex باقی می‌ماند اما فقط در صورتی مقداردهی می‌شود که ویژگی از نوع رنگ باشد. مدیریت نوع نمایش در UI از طریق فیلد DisplayType در جدول Attribute (مقادیر Text، Color، Dropdown) انجام می‌شود. اگر DisplayType برابر Color باشد، فیلد Hex پر می‌شود و در فرانت دایره رنگی رندر می‌گردد؛ در غیر این صورت نال می‌ماند و یک دکمه یا Badge نمایش داده می‌شود.

۶ عملیات پنل مدیریت: عدم اجبار در ترکیب‌ها و الگوی تولید ماتریس تنوع

در ادامه، سه پرسش رایج درباره عملیات ادمین و وارد کردن داده‌ها پاسخ داده می‌شود.

۱.۶ ۱. آیا فروشنده مجبور است به ازای هر رنگ تمام سایزها را وارد کند؟

قطعاً خیر. هیچ اجباری برای ایجاد تمام ترکیب‌های ممکن وجود ندارد. اگر از پارچه مشکی سایزهای S، M، L و XL تولید شده باشد اما از پارچه قرمز فقط سایزهای S و M، تنها ۶ سطر در جدول ProductVariants ساخته می‌شود. کاربر با انتخاب رنگ «قرمز»، دکمه‌های سایز L و XL را غیرفعال می‌بیند، زیرا دیتابیس اصلاً واریانتهای برای آن ۲. چرا قیمت در سطح SKU تکرار می‌شود؟

هر واریانت یک موجودیت فیزیکی مستقل برای فروش است. ممکن است امروز قیمت همه تنوع‌ها یکسان باشد، اما فردا پارت جدید قرمز گران‌تر تمام شود و فروشنده قیمت آن را جداگانه تغییر دهد. اگر قیمت در سطح SKU مستقل نباشد، دست فروشنده برای قیمت‌گذاری مجزا یا اعمال تخفیف کاملاً بسته می‌شود.

۲.۶ ۳. آیا ادمین باید قیمت‌های تکراری را دستی وارد کند؟

به هیچ وجه. فرم پنل مدیریت باید با الگوی Matrix Generator هوشمند طراحی شود:

۱. ادمین رنگ‌های [مشکی، قرمز] و سایزهای S، M، L را تیک می‌زند.
۲. فیلدهای «قیمت پیش‌فرض» (۴۰۰،۰۰۰) و «موجودی پیش‌فرض» (۱۰) را یک‌بار پر می‌کند.
۳. با دکمه «تولید ماتریس تنوع‌ها»، سیستم تمام ترکیب‌ها را در حافظه رندر می‌کند.
۴. ادمین ترکیب‌های ناموجود را حذف و قیمت‌های استثنا را ویرایش می‌کند و ذخیره می‌زند.

۳.۶ ۴. نکته تکمیلی: فیلد قیمت نال‌پذیر

اگر در اکثریت موارد قیمت تنوع‌ها یکسان است، می‌توان فیلد قیمت را نال‌پذیر (decimal? Price) تعریف کرد: اگر null بود، قیمت از Product.BasePrice خوانده می‌شود. با این حال، مستقل بودن کامل قیمت در ProductVariant، سرعت کوئری‌های فیلتر بر اساس ارزان‌ترین یا گران‌ترین کالا را در SQL به‌طور چشمگیری افزایش می‌دهد و از Join‌های پیچیده جلوگیری می‌کند.

۷ جمع‌بندی و توصیه‌های پیاده‌سازی

ساختار نهایی این معماری بر پایه پنج جدول است: Attributes و AttributeValues برای نگهداری ویژگی‌ها (رنگ با کد Hex و سایز بدون Hex)، Products برای موجودیت کلی محصول با فیلد PricingType، ProductVariants برای تعریف سطرهای انبارداری با قیمت و موجودی اختصاصی، و ProductVariantValues برای اتصال هر واریانت به ویژگی‌های متناظرش.

مزیت کلیدی این طراحی، پویایی کامل آن است: فروشگاه بدون نیاز به تغییر اسکیمای دیتابیس می‌تواند هر نوع محصول جدیدی (با هر ترکیب ویژگی) را در آینده به سیستم اضافه کند. وابستگی نمایشی بین ویژگی‌ها به فرانت‌اند سپرده می‌شود و قیمت‌گذاری مستقل در سطح SKU، دست فروشنده را برای قیمت‌گذاری و تخفیف باز می‌گذارد. توصیه می‌شود پیاده‌سازی پنل مدیریت با الگوی Matrix Generator انجام شود تا از ورود دستی و تکراری داده‌ها جلوگیری گردد.